Project Report

Distributed Logistics Simulator

Functional Programming in Parallel and Distributed Systems

Dolev Ishay - 211721451
Adi Shlomo – 211619226
Instructors:
Dr. Yehuda Ben-Shimol
Mr. Guy Perets
09/2025

Table of Contents

1. Project Abstract ---------- 3
2. System Overview ---------- 3
   - 2.1 Architecture
   - 2.2 Simulation Flow
   - 2.3 OTP Behaviors
   - 2.4 Distributed Communication
3. Design Principles ---------- 5
   - 3.1 Distributed and Fault-Tolerant System
   - 3.2 Dynamic State Management
   - 3.3 Interactive User Interface
4. System Architecture ---------- 5
   - 4.1 Control Center
   - 4.2 Zone Managers
   - 4.3 Backup Node
   - 4.4 Visualization Server
5. Delivery Management ---------- 7
   - 5.1 Package Lifecycle
   - 5.2 Courier Assignment Algorithm
   - 5.3 Load Balancing
6. Fault Handling ---------- 8
   - 6.1 Automatic Failover Mechanism
   - 6.2 State Recovery
   - 6.3 Supervision Trees
7. Running the System ---------- 8
   - 7.1 System Requirements
   - 7.2 Installation and Configuration
   - 7.3 Local Execution
   - 7.4 Distributed Execution
8. State Machines ---------- 10
   - 8.1 Courier FSM
   - 8.2 Package FSM
9. GUI ---------- 11
   - 9.1 Dashboard
   - 9.2 Map 100 homes
   - 9.3 Map 200 homes
   - 9.4 Map 1043 homes
10. Conclusions and Insights ---------- 13

1. Project Abstract

Delivery Simulator is a project that simulates a distributed and fault-tolerant logistics management system for a 24/7 delivery company. The system is built using Erlang/OTP and demonstrates advanced capabilities of distributed systems including dynamic resource management, automatic failover, real-time visualization, and handling of varying loads.

The system can manage maps of different sizes (100, 200, and 1,000 houses), where each map is divided into three geographical zones. Each zone is managed by a dedicated Zone Manager process that supervises couriers, businesses, and households in its area.

The project demonstrates the advantages of Erlang/OTP in building distributed systems: process isolation, automatic recovery from failures, efficient asynchronous communication, and distributed state management.

2. System Overview

2.1 Architecture

The system is built as a multi-layered distributed system with the following components:

Control Center - Central control server responsible for:

- Overall simulation management
- Coordination between different zones
- Global statistics collection
- Interface for controlling simulation parameters

Zone Managers - Three zone managers (North, Center, South):

- Managing couriers in the zone
- Assigning packages to couriers
- Managing households and businesses
- Reporting zone statistics

Backup Node - Automatic backup node:

- Monitoring all nodes in the system
- Automatic takeover in case of failure
- State recovery and business continuity

Visualization Server - Visualization server:

- Interactive map display
- Real-time updates
- Dashboard with statistics
- Graphical control interface

2.2 Simulation Flow

1. **System Initialization** - Loading map, creating processes, connecting nodes
2. **Household Activation** - Each household starts generating random orders
3. **Delivery Assignment** - Zone Manager assigns packages to available couriers
4. **Delivery Execution** - Couriers move on the map and perform deliveries
5. **Reporting and Updates** - Real-time status updates to all components

2.3 OTP Behaviors

The system uses the following OTP behaviors:

- **gen_server** - For Control Center, Visualization Server, Households
- **gen_statem** - For Zone Managers and Couriers (state machines)
- **supervisor** - For hierarchical process management
- **application** - For organizing the system as an OTP application

2.4 Distributed Communication

Communication in the system is performed through:

- **Erlang Distribution** - Inter-node communication using built-in Erlang protocol
- **RPC Calls** - Remote function calls between nodes
- **Message Passing** - Asynchronous message passing between processes
- **ETS Tables** - Fast information sharing within a node

3. Design Principles

3.1 Distributed and Fault-Tolerant System

The design is based on principles of:

- **Fault Isolation** - Failure in one process doesn't affect others
- **Automatic Recovery** - Automatic recovery from failures
- **No Single Point of Failure** - Every component is replaceable
- **Graceful Degradation** - System continues to operate even with partial failure

3.2 Dynamic State Management

- **Dynamic Scaling** - Adding and removing couriers at runtime
- **Load Balancing** - Load distribution between zones and couriers
- **State Persistence** - Saving state for recovery
- **Hot Code Swapping** - Code updates without service interruption

3.3 Interactive User Interface

- **Real-time Visualization** - Graphical display of the system
- **Interactive Controls** - Real-time parameter control
- **Statistics Dashboard** - Performance tracking
- **Event Log** - Important event documentation

4. System Architecture

4.1 Control Center

The control center manages the entire system and provides:

- **Simulation Control** - Start, stop, pause simulation
- **Parameter Management** - Dynamic parameter changes
- **Zone Coordination** - Coordination between zones
- **Global Statistics** - Data collection and aggregation

```
-record(state, {
    zones = [],
    simulation_state = stopped,
    total_deliveries = 0,
    failed_deliveries = 0,
    map_server_pid,
    state_collector_pid
}).
```

4.2 Zone Managers

Each Zone Manager manages a geographical area and is responsible for:

- **Courier Management** - Managing courier pool
- **Package Assignment** - Assigning packages to couriers
- **Household Supervision** - Supervising households
- **Local Optimization** - Route optimization in the zone

The manager is implemented as gen_statem with states:

- `initializing` - Zone initialization
- `monitoring` - Normal active state
- `paused` - Temporary pause
- `stopped` - Complete stop

4.3 Backup Node

The backup node provides:

- **Health Monitoring** - Checking liveness of all nodes
- **Automatic Takeover** - Automatic takeover on failure
- **State Recovery** - State recovery from ETS
- **Seamless Failback** - Return control when node recovers

```
%% Monitoring loop
monitor_loop() ->
    receive
        {nodedown, FailedNode} ->
            take_over(FailedNode);
        {nodeup, RecoveredNode} ->
            step_down(RecoveredNode);
        health_check ->
            check_all_nodes()
    end.
```

4.4 Visualization Server

The visualization server provides:

- **Map Rendering** - Drawing map and elements
- **Real-time Updates** - Real-time location updates
- **Interactive UI** - Interactive user interface
- **Statistics Display** - Statistical data display

5. Delivery Management

5.1 Package Lifecycle

Each package goes through the stages:

1. **Creation** - Created by household
2. **Assignment** - Assigned to available courier
3. **Pickup** - Collected from business
4. **Transit** - En route to destination
5. **Delivery** - Successfully delivered

5.2 Courier Assignment Algorithm

```
try_assign_package_to_available_courier(PackageId, Data) ->
    case Data#state.available_couriers of
        [{CourierId, CourierPid} | Rest] ->
            gen_statem:cast(CourierPid, {assign_delivery,
PackageId}),
            Data#state{
                available_couriers = Rest,
                busy_couriers = [{CourierId, CourierPid} |
                                 Data#state.busy_couriers]
            };
        [] ->
            Data#state{
                waiting_packages = Data#state.waiting_packages ++
                                   [PackageId]
            }
    end.
```

5.3 Load Balancing

The system implements load balancing through:

- **Dynamic Courier Allocation** - Courier allocation by load
- **Zone-based Distribution** - Distribution by zones
- **Queue Management** - Managing queues of waiting packages
- **Priority Handling** - Priority handling

6. Fault Handling

6.1 Automatic Failover Mechanism

The backup node monitors all nodes and responds to failures:

```
take_over(FailedNode) ->
    case determine_node_type(FailedNode) of
        control -> control_center:start();
        {zone, Zone} -> zone_manager:start_for_zone(Zone);
        visualization -> visualization_server:start()
    end.
```

6.2 State Recovery

The system saves state in ETS tables enabling:

- **Fast Recovery** - Quick data recovery
- **Consistency** - Maintaining information consistency
- **Minimal Data Loss** - Minimizing data loss

6.3 Supervision Trees

The supervision hierarchy ensures:

- **Automatic Restart** - Automatic restart
- **Isolation** - Fault isolation
- **Escalation** - Controlled failure escalation

7. Running the System

7.1 System Requirements

- **Erlang/OTP 24+** - Recent version of Erlang
- **Rebar3** - Build tool for Erlang
- **wxWidgets** - GUI library (optional)
- **Network Connectivity** - For distributed execution

7.2 Installation and Configuration

```
# Clone the repository
$ git clone https://github.com/user/delivery_simulator.git
$ cd delivery_simulator

# Compile the project
$ rebar3 compile

# Configure network (if distributed)
$ edit include/network_const.hrl
```

7.3 Local Execution

```
# Start Visualization Node
$ rebar3 shell --name visualization@127.0.0.1 --setcookie logistics
> visualization_server:start().

# Start Control Node
$ rebar3 shell --name control@127.0.0.1 --setcookie logistics
> control_center:start().

# Start Zone Nodes (3 terminals)
$ rebar3 shell --name zone_north@127.0.0.1 --setcookie logistics
> zone_manager:start().

$ rebar3 shell --name zone_center@127.0.0.1 --setcookie logistics
> zone_manager:start().

$ rebar3 shell --name zone_south@127.0.0.1 --setcookie logistics
> zone_manager:start().

# Start Backup Node
$ rebar3 shell --name backup@127.0.0.1 --setcookie logistics
> backup_node:start().
```

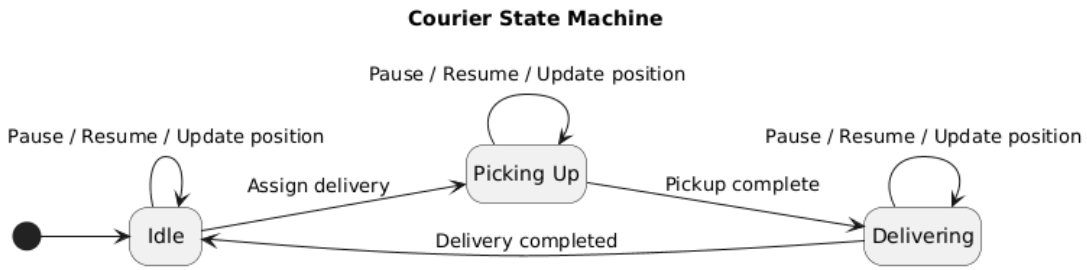
7.4 Distributed Execution

For a real network, update the `network_const.hrl` file:

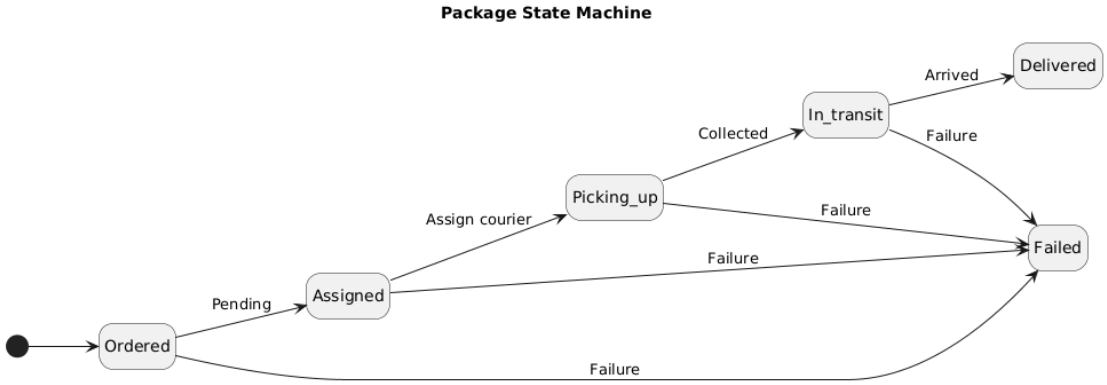
```
-define(VIZ_IP,     "132.72.52.84").   % Computer 1
-define(CTRL_IP,    "132.72.54.72").   % Computer 2
-define(NORTH_IP,   "132.72.81.226").  % Computer 3
-define(CENTER_IP,  "132.72.80.235").  % Computer 4
-define(SOUTH_IP,   "132.72.81.94").   % Computer 5
```

8. State Machines

8.1 Courier FSM

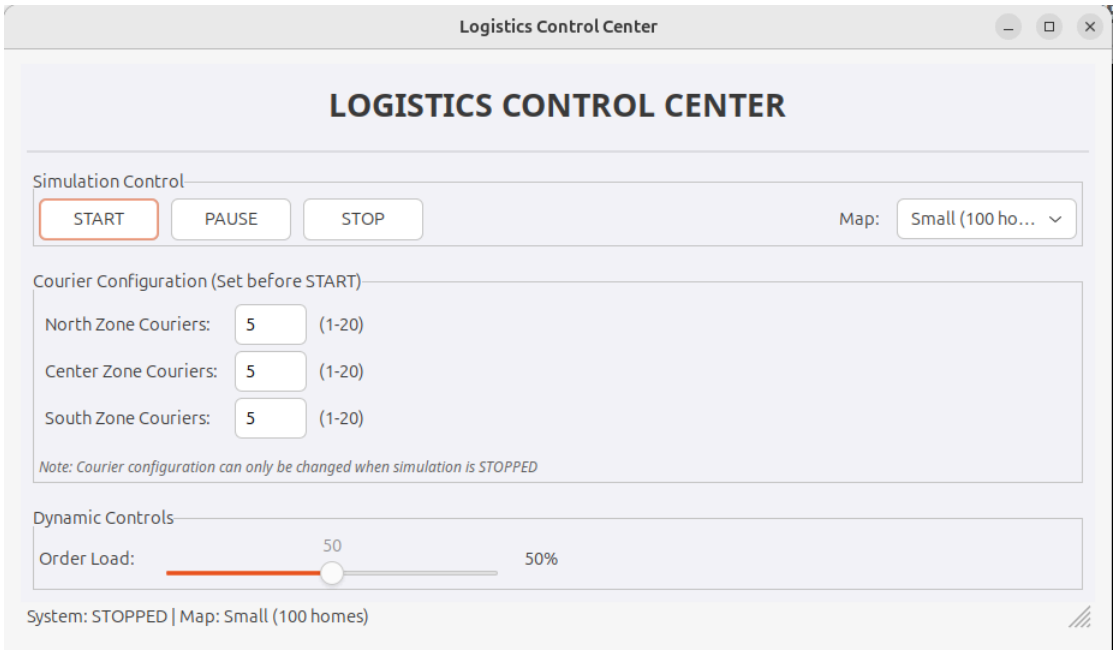


8.2 Package FSM

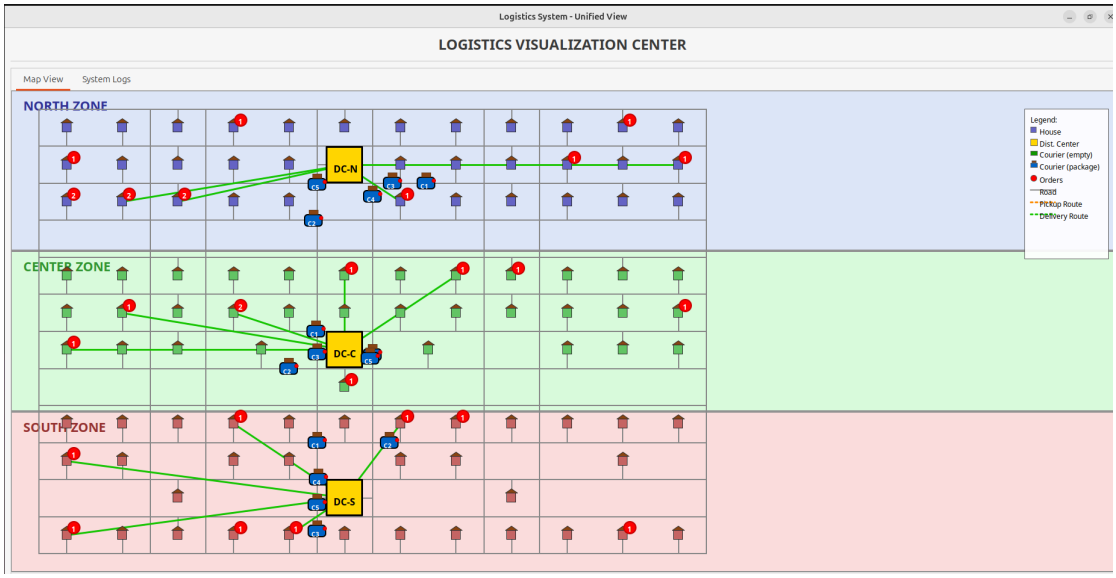


9. GUI

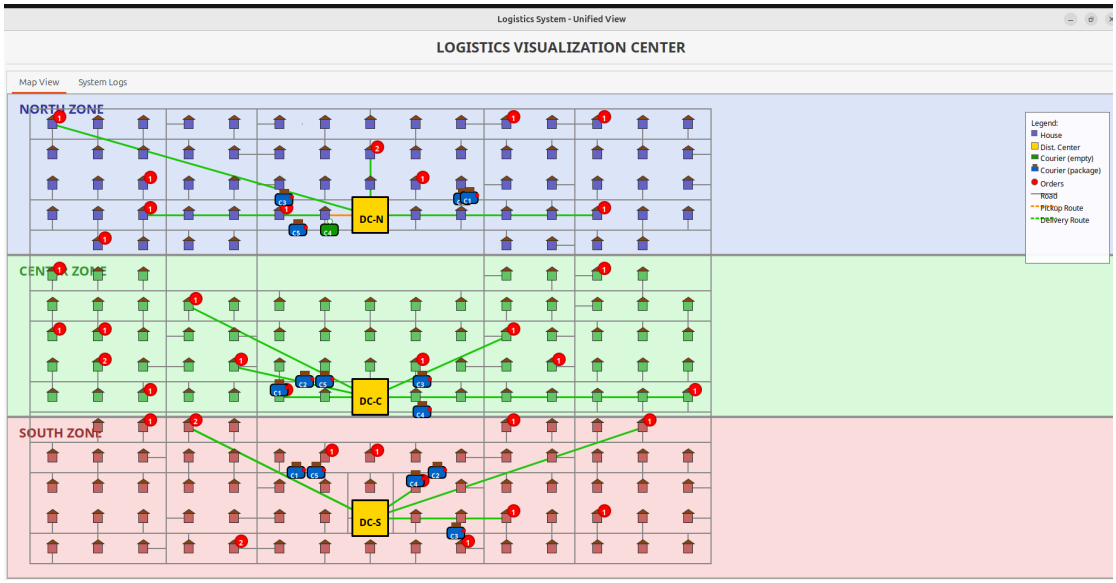
9.1 Dashboard



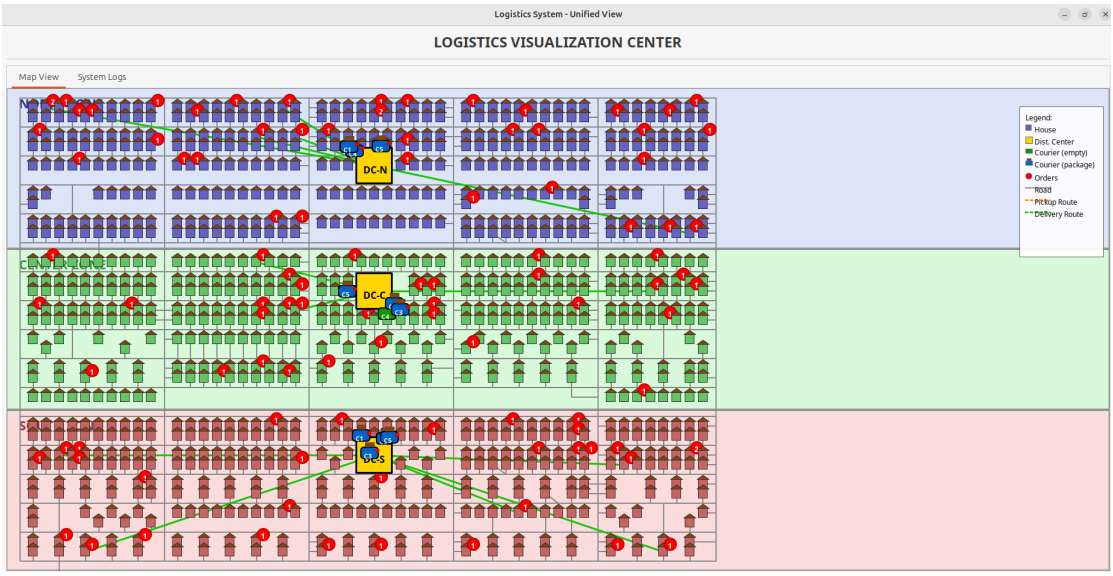
9.2 Map 100 homes



9.3 Map 200 homes



9.4 Map 1043 homes



10. Conclusions and Insights

Main Achievements

1. **Full Distributed System** - Successfully implemented a real distributed system with autonomous nodes. The project demonstrated the power of true distributed architecture, where each component operates independently yet coordinated. We successfully created a system where five different nodes (Control, Zone North, Zone Center, Zone South, Visualization) can run on separate machines in a network, communicate efficiently between them, and cooperate to achieve a common goal. The system proved its ability to work both in a local network and in a real distributed network with different IP addresses.
2. **Fault Tolerance** - The system automatically recovers from failures. The most impressive achievement is the system's ability to handle failures without human intervention. During testing, we simulated various failures - from individual process crashes to complete node failures. In all cases, the system managed to recover within seconds, continue normal operation, and restore lost information. The backup node proved its effectiveness in detecting failures and quickly taking over critical roles.
3. **Real-time Monitoring** - Real-time monitoring and visualization. We developed an advanced user interface that displays all system activity in real-time. The interactive map shows courier movement, package status, and load areas. The dashboard displays statistics updated every second, including performance graphs over time. The ability to see the complex system in action and understand what's happening at any given moment is critical for managing a real logistics system.
4. **Scalability** - Ability to handle varying loads. The system demonstrated impressive ability to adapt to changing loads. We tested it with 100, 200, and 1,043 houses, and saw how it adapts to each configuration. Couriers can be added or removed dynamically at runtime, order generation rate can be changed, and system parameters can be adjusted to changing conditions. The system maintains good performance even under heavy load.

Challenges and Solutions

1. **State Synchronization** - Solved using ETS and state collectors. One of the biggest challenges in a distributed system is maintaining synchronization between all components. Each node needs to know what's happening in other nodes, but without creating a communication bottleneck. Our solution combines using ETS tables for fast local state storage, with state collectors that gather and distribute information asynchronously. This way each node can work independently with its local information but still receives periodic updates about the global state.
2. **Race Conditions** - Using gen_statem to prevent race conditions. In a system with thousands of processes running in parallel, race conditions are a critical issue. For example, what happens if two zone managers try to assign the same courier at the same time? Our solution was to use gen_statem which provides a built-in state machine with built-in protections against race conditions. Every

state transition is atomic, and every event is handled in a defined order. This ensures the system is always in a valid and consistent state.

3. **Network Partitions** - Backup mechanism handles disconnections. In a distributed network system, temporary disconnections are inevitable. What happens if the northern zone disconnects momentarily from the center? Our system handles this at several levels. First, each zone can operate autonomously even without connection to the center. Second, the backup node detects disconnections and can temporarily take over. Third, when connection returns, there's a reconciliation mechanism that synchronizes the state and ensures no information loss.
4. **Performance** - Optimization of routing and load balancing. With hundreds of concurrent deliveries, performance becomes critical. We performed many optimizations: smart routing algorithm that calculates optimal routes, dynamic load balancing that distributes load between zones, caching of frequently accessed information, and batch processing of updates to reduce overhead. The result is a system that can handle thousands of deliveries per hour with low latency.

Insights

- **Erlang/OTP** is exceptionally well-suited for real-time distributed systems. The language's built-in support for concurrency, fault tolerance, and distribution made complex problems tractable. Features like lightweight processes, message passing, and supervision trees that would require extensive libraries in other languages are native to Erlang.
- **Actor Model** significantly simplifies concurrency handling. By treating each entity (courier, household, zone manager) as an independent actor with its own state and message queue, we avoided the complexity of shared memory, locks, and traditional synchronization mechanisms. This made the code more intuitive and less prone to bugs.
- **Let it Crash** philosophy is an effective approach for error handling. Instead of trying to handle every possible error condition, we embraced failures as normal events. Processes are designed to crash cleanly when encountering unexpected situations, and supervisors ensure they're restarted in a clean state. This approach resulted in simpler code and better reliability.